Question

Finding the Middle Element in a Linked List

Write a program to find the middle element of a singly linked list.
If the number of nodes is even, return the second middle element.

Solution

Approach (Two-Pointer Method)

- Use two pointers:
  - **Slow pointer** → moves one step at a time
  - **Fast pointer** → moves two steps at a time
- When the fast pointer reaches the end, the slow pointer will be at the middle.

C Program

```C/C++
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->data = data;
    newNode->next = NULL;
```

```c
    return newNode;
}

// Function to insert node at end
void insertEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}

// Function to find middle element
void findMiddle(struct Node* head) {
    struct Node *slow = head, *fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    if (slow != NULL)
        printf("Middle Element: %d\n", slow->data);
}

// Display list
void display(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
```

```c
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;

    insertEnd(&head, 10);
    insertEnd(&head, 20);
    insertEnd(&head, 30);
    insertEnd(&head, 40);
    insertEnd(&head, 50);

    printf("Linked List: ");
    display(head);

    findMiddle(head);

    return 0;
}
```

Output

Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
Middle Element: 30

Time & Space Complexity

- **Time Complexity:** O(n)
- **Space Complexity:** O(1)

Question

Write a C program to remove duplicates from a single unsorted linked list.

Solution

Approach (Using Two Pointers – No Extra Space)

- Traverse the list using a pointer `current`.
- For each node, use another pointer `temp` to check the remaining list.
- If a duplicate node is found, delete it.
- This method does not use extra memory but takes more time.

C Program

```C/C++
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Create new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

```c
// Insert at end
void insertEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}

// Remove duplicates
void removeDuplicates(struct Node* head) {
    struct Node *current, *temp, *prev;
    current = head;
    while (current != NULL) {
        prev = current;
        temp = current->next;
        while (temp != NULL) {
            if (temp->data == current->data) {
                prev->next = temp->next;
                free(temp);
                temp = prev->next;
            } else {
                prev = temp;
                temp = temp->next;
            }
        }
        current = current->next;
    }
}

// Display list
void display(struct Node* head) {
```

```c
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main
int main() {
    struct Node* head = NULL;
    insertEnd(&head, 10);
    insertEnd(&head, 20);
    insertEnd(&head, 10);
    insertEnd(&head, 30);
    insertEnd(&head, 20);
    insertEnd(&head, 40);

    printf("Original List: ");
    display(head);

    removeDuplicates(head);

    printf("After Removing Duplicates: ");
    display(head);

    return 0;
}
```

Output

Original List: 10 -> 20 -> 10 -> 30 -> 20 -> 40 -> NULL
After Removing Duplicates: 10 -> 20 -> 30 -> 40 -> NULL

Complexity

- **Time Complexity:** $O(n^2)$
- **Space Complexity:** $O(1)$

Question

Write a C program for Union and Intersection of two linked lists.

Solution

Concept

- **Union** → All unique elements from both lists
- **Intersection** → Common elements present in both lists

Approach

1. Traverse both linked lists
2. For **Union**:
    - Add all elements from both lists
    - Avoid duplicates
3. For **Intersection**:
    - Compare elements of both lists
    - Store only common elements

C Program

```C/C++
#include <stdio.h>
#include <stdlib.h>

// Node structure
```

```c
struct Node {
    int data;
    struct Node* next;
};

// Create node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Insert at end
void insertEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}

// Check if element exists
int exists(struct Node* head, int data) {
    while (head != NULL) {
        if (head->data == data)
            return 1;
        head = head->next;
    }
    return 0;
}
```

```c
// Union of two lists
struct Node* unionList(struct Node* head1, struct Node* head2) {
    struct Node* result = NULL;
    while (head1 != NULL) {
        if (!exists(result, head1->data))
            insertEnd(&result, head1->data);
        head1 = head1->next;
    }
    while (head2 != NULL) {
        if (!exists(result, head2->data))
            insertEnd(&result, head2->data);
        head2 = head2->next;
    }
    return result;
}

// Intersection of two lists
struct Node* intersectionList(struct Node* head1, struct Node*
head2) {
    struct Node* result = NULL;
    while (head1 != NULL) {
        if (exists(head2, head1->data) && !exists(result,
head1->data))
            insertEnd(&result, head1->data);
        head1 = head1->next;
    }
    return result;
}

// Display list
void display(struct Node* head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
```

```
    printf("NULL\n");
}

// Main function
int main() {
    struct Node *head1 = NULL, *head2 = NULL;
    insertEnd(&head1, 10);
    insertEnd(&head1, 20);
    insertEnd(&head1, 30);
    insertEnd(&head2, 20);
    insertEnd(&head2, 30);
    insertEnd(&head2, 40);
    printf("List 1: ");
    display(head1);
    printf("List 2: ");
    display(head2);
    struct Node* uni = unionList(head1, head2);
    struct Node* inter = intersectionList(head1, head2);
    printf("Union: ");
    display(uni);
    printf("Intersection: ");
    display(inter);
    return 0;
}
```

Output

List 1: 10 -> 20 -> 30 -> NULL
List 2: 20 -> 30 -> 40 -> NULL
Union: 10 -> 20 -> 30 -> 40 -> NULL
Intersection: 20 -> 30 -> NULL

Complexity

- **Union:** O(n × m)
- **Intersection:** O(n × m)
- **Space:** O(n)

Question

Write a C program to reverse a linked list.

Solution

Approach (Iterative Method)

- Use three pointers:
  - `prev` → initially NULL
  - `current` → points to head
  - `next` → stores next node
- Reverse the `next` pointer of each node step by step

C Program

```C/C++
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Create node
```

```c
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Insert at end
void insertEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}

// Reverse linked list
struct Node* reverseList(struct Node* head) {
    struct Node *prev = NULL, *current = head, *next = NULL;
    while (current != NULL) {
        next = current->next;   // store next
        current->next = prev;   // reverse link
        prev = current;         // move prev forward
        current = next;         // move current forward
    }
    return prev; // new head
}

// Display list
void display(struct Node* head) {
    while (head != NULL) {
```

```
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

// Main
int main() {
    struct Node* head = NULL;
    insertEnd(&head, 10);
    insertEnd(&head, 20);
    insertEnd(&head, 30);
    insertEnd(&head, 40);
    printf("Original List: ");
    display(head);
    head = reverseList(head);
    printf("Reversed List: ");
    display(head);
    return 0;
}
```

Output

Original List: 10 -> 20 -> 30 -> 40 -> NULL
Reversed List: 40 -> 30 -> 20 -> 10 -> NULL

Complexity

- **Time Complexity:** O(n)
- **Space Complexity:** O(1)

Question

Write a C program to check if two linked lists are equal.

Solution

Approach

Two linked lists are equal if:

1. They have the **same number of nodes**
2. Each corresponding node has the **same data**

👉 Traverse both lists simultaneously and compare data at each node.

C Program

C/C++

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Create node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

```c
// Insert at end
void insertEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}

// Check if lists are equal
int areEqual(struct Node* head1, struct Node* head2) {
    while (head1 != NULL && head2 != NULL) {
        if (head1->data != head2->data)
            return 0; // not equal
        head1 = head1->next;
        head2 = head2->next;
    }
    if (head1 == NULL && head2 == NULL) line_break
        return 1;
    return 0;
}

// Display list
void display(struct Node* head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}
```

```c
// Main
int main() {
    struct Node *head1 = NULL, *head2 = NULL;
    insertEnd(&head1, 10);
    insertEnd(&head1, 20);
    insertEnd(&head1, 30);
    insertEnd(&head2, 10);
    insertEnd(&head2, 20);
    insertEnd(&head2, 30);
    printf("List 1: ");
    display(head1);
    printf("List 2: ");
    display(head2);
    if (areEqual(head1, head2))
        printf("Lists are Equal\n");
    else
        printf("Lists are Not Equal\n");
    return 0;
}
```

Output

List 1: 10 -> 20 -> 30 -> NULL

List 2: 10 -> 20 -> 30 -> NULL

Lists are Equal

Complexity

- **Time Complexity:** O(n)
- **Space Complexity:** O(1)